# The Ledger: Addendum A1 to v10.3

StatesHome — Where Unit State Lives

R. Delloye

April 2026 — Addendum A1 to v10.3

**Abstract**

Section 7 of the Ledger v10.3 specification leaves one question open: is the state of a unit attached to the wallet or to the unit? Four instruments answer it by forcing distinct keyings that no single attachment can carry — a future whose state depends on past trades, a managed account whose state is inherently per-client, a systematic strategy that itself trades futures, and an instrument registered but never traded. Unit state therefore lives in three maps, each with its own mutation discipline: an immutable, versioned *ProductTerms* keyed by unit; a *UnitStatus* keyed by unit and shared across holders — a materialised projection of the log, a cached view the log can always rebuild, whose every change is caused by a logged event and which replay reconstructs exactly; and a *PositionState* keyed by (wallet, unit). There is no wallet-keyed state sector: every per-wallet economic fact is keyed by a mandate or strategy unit and so lives in *PositionState*. Twelve disciplines on these maps, collected in §5, render seven of the ten core invariants of v10.3 §11 unrepresentable rather than merely tested. This addendum supersedes the phrasing at v10.3 line 1034 and line 2287.

## Contents

# 1 The Question

Should the state of a unit be attached to the wallet or to the unit?

Four instruments constrain the answer, and no two constrain it the same way:

1. a future whose state depends on past transactions;
2. a managed account, whose state is inherently attached to a wallet;
3. a systematic (QIS) strategy that trades futures;
4. an instrument that exists in the universe but has not been traded yet.

Each is worked in §4. Each forces a part of the schema, and the conditions that govern the schema are introduced where the instrument forces them, then collected in §5.

# 2 Notation

The framework primitives — wallet, unit, holding, move, transaction, conservation — are those of v10.3. The symbols below are fixed for the remainder of this document.

| **Symbol / term** | **Meaning** |
|---|---|
| $\mathcal{W}$, $w$ | The set of wallets; a wallet. |
| $\mathcal{U}$, $u$ | The set of units; a unit. A *unit* is any object that can be held and conserved: an instrument, and also a mandate or a strategy contract (§4.2, §4.3). |
| $h(w, u)$ | The signed quantity of unit $u$ held in wallet $w$, in exact minor units. *Conservation* is the framework invariant $\sum_{w \in \mathcal{W}} h(w, u) = 0$ for every $u$, with $h(w, u) = 0$ for all but finitely many $w$. |
| Map$[K, V]$ | A finite partial function $K \rightharpoonup V$. Its accessor returns Option$[V]$. |
| Option$[V]$ | Some $v$ (a value $v$ is present) or None (no value). |
| NonEmptyList$[T]$ | A list of one or more elements of $T$. |
| total on $S$ | Defined for every key in $S$. |
| registration-total | Total on the set of *registered* units, undefined elsewhere. |
| append-only | The only write is append; no key is overwritten in place and none is deleted. |
| monotone carrier | A map whose domain only grows: once a key is inserted it is never removed. |
| $\Delta f(w, u)$ | The change an event proposes to field $f$ of *PositionState*$[w, u]$; $\Delta f(w, u) = 0$ for every wallet the event does not touch. |
| conserved field | A *PositionState* field that nets to zero across wallets: `accumulated_cost`, and `balance`. `balance` is a second conserved field, moved by a transfer, carried only by the reference (§12) to exercise the C11 per-field-writer discipline with a canonical writer distinct from that of `accumulated_cost`; it is neither the framework holding $h(w, u)$ nor an economic datum of the §3 inventory. `hwm` and `entry_nav` are non-conserved. |
| flat | A position whose conserved fields are zero. |

| | |
|---|---|
| $0_P$ | The *PositionState* with every field zero (`zeroP`, §12). A first-touch row is $0_P$; a flat row need not be, since it may retain a non-conserved field (a crystallised `hwm`, an `entry_nav`). |
| *StateDelta* | The change one event proposes, for a single unit, across the three maps; conservation-checked and atomic (C2, C3). |
| $u_{\text{MA}}$ | A managed-account mandate, taken as a unit (§4.2). |
| $u_{\text{QIS}}$ | A strategy contract, taken as a unit (§4.3). |
| $u_{\text{ES}}, u_{\text{NQ}}, u_{\text{YM}}$ | Futures the strategy trades. |
| $u_{\text{bench}}$ | A benchmark, taken as a unit. |
| $w_{\text{QIS}}$, $w_C$ | The strategy's own wallet; a client wallet. |
| $u_{\text{old}} \to u_{\text{new}}$ | The pair an identity-breaking amendment allocates (§4.4). |

The holding symbol $h(w, u)$ is fixed here for this document; where v10.3 fixes a symbol for quantity, that symbol governs.

## 3 The Answer

State attaches to three objects, each with its own map and its own mutation discipline.

```
ProductTerms  : Map[UnitId, NonEmptyList[TermsVersion]]   -- immutable, append-only,
    versioned; reg-total
UnitStatus    : Map[UnitId, UnitStatus]                   -- projection of the log,
    overwritten in place as a read cache; shared,
                                                          -- read identically by every
    holder; registration-total --- every change
                                                          -- caused by a logged event,
    rebuilt exactly by replay
PositionState : Map[(WalletId, UnitId), PositionState]    -- per (holder, unit); monotone,
    Option accessor
```

Each map's value type carries its sector's name by intent: a *UnitStatus* entry is one `UnitStatus` record, a *PositionState* entry one `PositionState` record. The reference names the maps themselves `ledgerUS` and `ledgerPS` to keep the two apart (§12).

**The UnitStatus discipline, stated once.** UnitStatus holds one value per unit, shared — read identically by every holder. Its value changes over time, but UnitStatus is not a separate source of truth: the immutable event log is. Every change is caused by a logged event; the stored value is overwritten in place only as a read cache. Replaying the events up to any point rebuilds the exact value that held then, so nothing is lost by overwriting and there is no other way to change the value. A value exists from registration onward. Concretely, the stored value always equals what replaying the unit's events produces, and no part of the system writes it except through a logged event; in the reference (§12) this holds by construction, since the only writers of *UnitStatus* are `register` and `applyDelta` (both folds of events) and the `Ledger` constructor is unexported, so no out-of-band `setStatus` exists. Externally-sourced inputs — a settlement price, the current benchmark level — enter only as a logged observation event, so the fold stays pure at the boundary.
A fourth map carries identity, not state, and holds no financial value:

```
WalletRegistry : Map[WalletId, WalletMetadata]            KYC, permissions, audit
    cursor -- NOT state
```

The *audit cursor* in *WalletMetadata* is the position of the wallet's reader in the event stream. *WalletRegistry* carries no economic quantity and participates in no conservation law.

There is no wallet-keyed state map. Call such a hypothetical map the *W-sector*: a Map[WalletId, WalletState holding per-wallet economic facts. §4.2 shows that every candidate datum for the $W$-sector is

keyed by a mandate or strategy unit, hence lives in *PositionState*; the $W$-sector is empty of economic state and is not built. The three maps are forced, and only three are forced, by §6.

**Home of each datum.**

| Key | Home | Examples |
|---|---|---|
| $u$, immutable | *ProductTerms* | multiplier, currency, expiry, clearinghouse, strike, ISIN, fee schedule, mandate text, benchmark identity, index methodology |
| projection of the log, overwritten in place as a read cache; shared, read identically by every holder; registration-total — every change caused by a logged event, rebuilt exactly by replay | *UnitStatus* | `lifecycle_stage`, `last_settlement_price`, `last_settlement_date`, `current_weights`, `nav_index`, `triggered_barrier`, `superseded_by` |
| $(w, u)$ | *PositionState* | `accumulated_cost`, `ccp_binding`, per-position OTC lifecycle, `entry_nav`, `hwm`, `accrued_{mgmt,perf}_fee`, `benchmark_nav_at_inception`, `mandate_breach_flags` |

`balance` is absent from this inventory by intent: it is the reference's demonstrative second conserved field (§2), exercising the per-field-writer discipline, not an economic datum of the schema.

# 4 The Four Instruments

Each condition is introduced at the instrument that forces it and stated once. The labels C1–C12 are stable tags, not a sequence: they follow the §5 index rather than order of first appearance, so the first met below is C2. They are load-bearing in §9 and §12. Parenthetical references to signals S1–S4 point to the labelled expressibility notes in the reference (§12).

## 4.1 A Future Whose State Depends on Past Transactions

Take the E-mini S&P future. Its data divide by who may read them and how often they change.

| Field | Home | Reason |
|---|---|---|
| `multiplier`, `currency`, `expiry`, `clearinghouse`, `exchange`, `product_id` | *ProductTerms*$[u_{\mathrm{ES}}]$ | Immutable. The CME contract and the ICE contract are *distinct units*; the clearinghouse is a term of the unit, not a per-wallet field (this replaces v10.3 line 1168). |
| `lifecycle_stage`, `last_settlement_price`, `last_settlement_date` | *UnitStatus*$[u_{\mathrm{ES}}]$ | One settlement price per contract, read identically by every holder. |

| | | |
|---|---|---|
| `accumulated_cost` (`ac`) | *PositionState*$[w, u_{\text{ES}}]$ | Two wallets holding the same contract carry different `ac`. |

**Why per-(wallet, unit) is forced.** Two wallets hold the same contract with different accumulated cost. Substituting one wallet's position for the other's changes `ac`; any $u$-only keying identifies the two and loses the difference. So `ac` is keyed by $(w, u)$, and *PositionState* exists.

**Conservation of `ac`.** `accumulated_cost` is a conserved field (§2): conservation as defined for $h$ extends to it, $\sum_{w \in \mathcal{W}} \texttt{ac}(w, u) = 0$ for every $u$. No map layout enforces this by types — a refinement type on a sum of decimals is not free in any production language. It is enforced one level up, at the event handler.

**Condition 4.1** (**C2**: handler-level conservation)**.** *Every event handler — `Trade`, `SettleVM`, `CorporateAction`, `QISRebalance`, `MandateAmend` — emits a `StateDelta` whose every conserved field satisfies*

$$\sum_{w \in \mathcal{W}} \Delta f(w, u) = 0$$

*structurally, by event class. The proof obligation per class covers the two-leg trade ($\Delta f = +q$ and $-q$), the $K$-leg trade ($K$ balanced legs), the variation-margin fan-out (each wallet reset to target, cash legs offsetting), and the vacuous case of zero touched wallets (the empty sum is 0). Induction over the event stream lifts the per-event identity to the global invariant.*

For the future: a `Trade` contributes buyer $\Delta\texttt{ac} = +q$ and seller $\Delta\texttt{ac} = -q$, summing to 0; a `SettleVM` resets each wallet to its target with offsetting cash legs, summing to 0. The worked legs are the proof.

**What is not state.** `first_touch_date` is derived from the event log on demand, not stored. Caching it in *PositionState* would make a replay disagree with itself under a back-dated correction, because the cached value would reflect the pre-correction order while the fold reflects the corrected one. The field is therefore absent from *PositionState*.

**Settled positions persist.** A closed-out position keeps its row, flat, rather than being deleted. The row is read after close-out for tax reporting, wash-sale lookback, and trade reconstruction. Two disciplines on *PositionState* make this precise; they are orthogonal — one constrains the accessor's type, the other constrains storage — and both are required.

**Condition 4.2** (**C1**: Option accessor and monotone carrier)**.** (a) Option accessor. *`position(w,u)` returns* Option[*PositionState*]. None *means* this wallet has never held this unit*;* Some($p$) *with $p$ flat means* held once, now flat*. The two readings are distinct and both load-bearing — variation-margin settlement, wash-sale lookback, and record-date entitlement each act on the difference — so* None *and* Some($p$) *are never collapsed.*
(b) Monotone carrier. *Once a row is created it is never removed; close-out leaves a flat row. Replay is then a literal fold: `apply_all(events)` is independent of where checkpoints are cut, because the key set is stable across cuts, and conservation is a single pass over that stable key set.*

The two halves are independent: (a) is a property of the return type, (b) a property of the domain. Neither implies the other; the design carries both.

**A canonical writer set per field.** `ac` is written only by settle and trade handlers; a fee field is written only by the fee handler; an entry-NAV field is written only at subscription. Concentrating each field's writers removes cross-handler interference as a possibility rather than a thing to test.

**Condition 4.3** (**C11**: per-field canonical writer)**.** *Each* PositionState *field is tagged with its canonical writer set — the closed set of field-writers permitted to mutate it:* `ac`→*settle/trade;* `balance`→*transfer;* `hwm`→*fee_crystallise;* `entry_nav`→*subscribe. A write by any field-writer outside that set is a type error at the writer's authorship site; the tag is erased once writes share one delta row, so the guarantee binds at authorship, not at the stored row (§12, signal S3). These field-writers (settle, trade, transfer, fee_crystallise, subscribe) are a different axis from the event classes of C2 (*`Trade`*,* `SettleVM`*,* `CorporateAction`*,* `QISRebalance`*,* `MandateAmend`*): one event class drives one or more field-writers, and the two name-sets are not meant to coincide.*

## 4.2 A Managed Account — The "Inherently Wallet-Attached" Case

The managed account is the case that appears to demand a $W$-sector: its state is the client's, not the instrument's. It does not, because *the mandate is a unit.* v10.3 §6 already places mandates in $\mathcal{U}$ ("the managed-account smart contract", "CSA margin as a wallet-level smart contract"). The mandate $u_{\mathrm{MA}}$ is issued by the manager and held by the client, and so conserves like any issued unit:

$$h(w_{\mathrm{manager}}, u_{\mathrm{MA}}) = -1, \qquad h(w_{\mathrm{client}}, u_{\mathrm{MA}}) = +1, \qquad \sum_{w \in \mathcal{W}} h(w, u_{\mathrm{MA}}) = 0.$$

With $u_{\mathrm{MA}}$ a unit, every field that looked per-wallet relocates to one of the three maps:

| **Field** | **Home** |
|---|---|
| Mandate text, fee schedule, benchmark identity, max position limits | $\mathit{ProductTerms}[u_{\mathrm{MA}}]$ |
| HWM hurdle methodology, crystallisation frequency | $\mathit{ProductTerms}[u_{\mathrm{MA}}]$ |
| HWM *value* (client-specific), `hwm_date` | $\mathit{PositionState}[w_{\mathrm{client}}, u_{\mathrm{MA}}]$ |
| Accrued mgmt / perf fee, mandate breach flags, subscription / redemption cursor | $\mathit{PositionState}[w_{\mathrm{client}}, u_{\mathrm{MA}}]$ |
| Benchmark NAV at this wallet's inception | $\mathit{PositionState}[w_{\mathrm{client}}, u_{\mathrm{MA}}]$ |
| Current benchmark level (shared, captured as a logged observation event from the index source) | $\mathit{UnitStatus}[u_{\mathrm{bench}}]$ |

**Condition 4.4** (**C12**: $W$-sector collapse)**.** *All per-(wallet, mandate/strategy) economic state lives at* PositionState$[w, u_{\mathrm{MA}}]$ *or* PositionState$[w, u_{\mathrm{QIS}}]$*. No flat per-wallet scalar holds economic state; the empty $W$-sector is enforced by schema.*

**The mandate-as-unit is not a sentinel.** A rejected alternative (design C, §8) inserts a sentinel unit with no issuer, no holder, and no conservation partner, to carry wallet state. That sentinel breaks $\sum_w h(\cdot) = 0$ by fiat. $u_{\mathrm{MA}}$ is the opposite: a real contract with a real issuer (the manager) and a real holder (the client), conserving by the standard issuance law above. The collapse of the $W$-sector costs no conservation guarantee.

**Two mandates, two rows.** A client holding a base mandate and an overlay mandate on the same wallet carries two rows, $\mathit{PositionState}[w_{\mathrm{client}}, u_{\mathrm{MA,base}}]$ and $\mathit{PositionState}[w_{\mathrm{client}}, u_{\mathrm{MA,overlay}}]$, each with its own HWM, fees, and breach flags. A flat per-wallet scalar would have identified them. This is the direct reason C12 keys on the mandate unit and not on the wallet.

### 4.3 A Strategy That Trades Futures

A QIS strategy is itself a tradable unit, $u_{\text{QIS}}$, and it also holds futures in its own wallet. Five keyings coexist, and no two share a map:

```
u_QIS        the strategy itself (a tradable unit)
u_ES, u_NQ   the futures the strategy trades
u_MA,C       the mandate under which client C holds QIS (if any)
w_QIS        the strategy's own wallet (where its futures positions live)
w_C          a client wallet
```

| State | Home |
|---|---|
| Strategy terms (vol target, barrier, universe, share-class index start) | *ProductTerms*$[u_{\text{QIS}}]$ |
| Strategy state (`last_rebalance_date`, `current_weights`, `cumulative_return`, `nav_index`, `vol_realised`, `triggered_barrier`), shared across all holders | *UnitStatus*$[u_{\text{QIS}}]$ |
| Futures the strategy holds (`accumulated_cost`, ccp) | *PositionState*$[w_{\text{QIS}}, u_{\text{ES}}]$, $[w_{\text{QIS}}, u_{\text{NQ}}]$, $[w_{\text{QIS}}, u_{\text{YM}}]$ |
| Per-client subscription (`entry_nav`, `hwm`) | *PositionState*$[w_C, u_{\text{QIS}}]$ |
| Per-client mandate overlay (if any) | *PositionState*$[w_C, u_{\text{MA,C}}]$ |

**Where the HWM lives.** A client holding two strategies carries two high-water marks. Any wallet-only keying identifies them, exactly as in §4.2. The HWM therefore lives at *PositionState*$[w_C, u_{\text{QIS}}]$, or *PositionState*$[w_C, u_{\text{MA,C}}]$ when a mandate wrapper is in force.

**The rebalance is one atomic event over several units.** A barrier hit or monthly rebalance updates *UnitStatus*$[u_{\text{QIS}}]$ (new weights, and a new lifecycle stage if the barrier breaks) *and* the strategy's *PositionState*$[w_{\text{QIS}}, u_{\text{ES}}]$, $[w_{\text{QIS}}, u_{\text{NQ}}]$, $[w_{\text{QIS}}, u_{\text{YM}}]$ rows (the trades on the underlying futures). These lie on different units; the event is therefore one `StateDelta` per unit, applied together. A reader must never observe one applied and another not.

**Condition 4.5** (**C3**: atomic StateDelta)**.** *A `StateDelta` carries, for a single unit, the changes to* ProductTerms*,* UnitStatus*, and* PositionState*. It applies in full or not at all; a partially applied delta is not a representable state.*

`validate` discharges conservation for one unit across wallets ($\sum_w \Delta f(w,u) = 0$). An event touching several units — a rebalance trading `ES`, `NQ`, and `YM`, or a future bought against cash — is one validated `StateDelta` per unit; per-unit conservation composes to event-level conservation, and the group applies all-or-nothing as a single fold (§12, signal S1).

**Reads are scoped; exports flow through UnitStatus.** A strategy publishes its state to clients through *UnitStatus*$[u_{\text{QIS}}]$, the shared sector. A client's view of another client's overlay is not a read it is permitted to make.

**Condition 4.6** (**C4**: capability-scoped reads)**.** *Reads are capability-scoped. A cross-*$(w, u_{\text{MA}})$ *overlay read is forbidden. Strategy exports flow only through* UnitStatus*.*

**Wind-down.** Setting *UnitStatus*$[u_{\text{QIS}}]$.`lifecycle_stage` = `CLOSED` propagates to clients as a NAV-strike redemption. The *PositionState*$[w_C, u_{\text{QIS}}]$ rows are retained, flat, by C1(b); they hold the audit trail and the final HWM for tax reporting.

### 4.4 An Instrument Registered but Not Yet Traded

An untraded option exercises the registration and lifecycle paths with no positions present.

- *ProductTerms*$[u]$ is present at registration.
- *UnitStatus*$[u]$ is present and fully initialised at registration with product-declared defaults (e.g. `lifecycle_stage = REGISTERED`, `last_settlement_price = None`); `unit_status(u)` is total. `REGISTERED` denotes a unit recorded in and known to the ledger before any settlement — an OTC option, a bespoke mandate, or an internal strategy reaches it exactly as an exchange instrument does — not admission to an exchange.
- *PositionState*$[w, u]$ has no rows; `position(w,u)` is None for every $w$ until first touch (C1(a)).
- *WalletRegistry* is unaffected.

**Condition 4.7** (**C7**: ProductTerms registration-total)**.** ProductTerms *is registration-total: its first version is written at registration, so* `product_terms(u)` *is defined for every registered* $u$*.*

**Condition 4.8** (**C5**: UnitStatus registration-total)**.** UnitStatus *is registration-total: product-declared defaults are written at Unit Store registration, so* `unit_status(u)` *is defined for every registered* $u$*.*

**Lifecycle with no holders.** An untraded option moves `REGISTERED` → `ACTIVE` → `EXPIRED` entirely through *UnitStatus*, creating no *PositionState* row. The handlers fire on an empty holder set and discharge conservation vacuously.

**Condition 4.9** (**C9**: vacuous conservation on zero-holder units)**.** *A handler on a unit with no holders discharges* $\sum_w \Delta f = 0$ *vacuously; the per-class proof obligation of C2 includes the empty case. A handler that divides by holder count — the* `dividend_per_share / len(holders)` *bug class — is excluded, because conservation sums deltas and never divides by a count that may be zero.*

**Registration is once only.**

**Condition 4.10** (**C10**: no re-registration)**.** *Re-registering an existing unit id is a hard error, never a silent reset of its state.*

**Amendment has two tracks.** A bond amendment that preserves fungibility (coupon step-up, eligible-collateral change, a fee tweak within a declared band) appends a new `TermsVersion` to *ProductTerms*$[u]$; existing *PositionState* rows are untouched. An amendment that breaks fungibility (new ISIN, benchmark swap, restructuring) allocates a fresh $u_{\text{new}}$ and stamps `superseded_by`$(u_{\text{old}} \to u_{\text{new}})$ in *UnitStatus*. Moving holders from $u_{\text{old}}$ to $u_{\text{new}}$ is a *separate* paired-issuance event, not part of the amendment: a burn on $u_{\text{old}}$ and a mint on $u_{\text{new}}$, each conserving on its own unit, applied together. A single-unit `StateDelta` cannot span the two units (§12, signal S1). The track is chosen by a product-declared total predicate,

`is_fungibility_preserving` : *ProductTerms*$\times$*TermsAmendment* $\to$ {`Preserving`, `Breaking`}.

**Condition 4.11** (**C6**: ProductTerms append-only)**.** ProductTerms *is a* NonEmptyList[*TermsVersion*]*, versioned and append-only; no version is mutated in place.*

**Condition 4.12** (**C8**: two-track amendment)**.** *The fungibility predicate decides the track:* `Preserving` *appends a* `TermsVersion` *to the same unit;* `Breaking` *allocates a fresh* $u$ *and records* `superseded_by`*. The predicate is total and testable.*

# 5 The Twelve Conditions

Each condition is defined once, at the instrument that forces it. This index gives the label, its content in one line, and where it is established.

| | **Content** | **Defined** |
|---|---|---|
| C1 | Option accessor and monotone carrier, both. | §4.1 |
| C2 | Handler-level conservation, per event class, vacuous base case included. | §4.1 |
| C3 | Atomic `StateDelta` across all three maps; partial application unrepresentable. | §4.3 |
| C4 | Capability-scoped reads; cross-overlay reads forbidden; exports through *UnitStatus*. | §4.3 |
| C5 | *UnitStatus* registration-total; product-declared defaults at registration. | §4.4 |
| C6 | *ProductTerms* versioned, append-only; no in-place mutation. | §4.4 |
| C7 | *ProductTerms* registration-total; first write at registration. | §4.4 |
| C8 | Two-track amendment by a total fungibility predicate (Preserving / Breaking). | §4.4 |
| C9 | Zero-holder handlers discharge $\sum = 0$ vacuously; empty case in the proof obligation. | §4.4 |
| C10 | Re-registration is a hard error, never a silent reset. | §4.4 |
| C11 | Each *PositionState* field has a canonical writer set; a writer outside it is a type error at authorship (erased at the row, S3). | §4.1 |
| C12 | All per-(wallet, mandate/strategy) economic state at *PositionState*$[w, u_{\mathrm{MA}}]/[w, u_{\mathrm{QIS}}]$; $W$-sector empty by schema. | §4.2 |

# 6 Why Exactly Three Maps

Three independent constraints each break a distinct map; removing any one of the three breaks the corresponding constraint.

1. **Distinct per-holder state under the same contract.** Two wallets holding the same contract carry distinct accumulated cost and lifecycle state (§4.1). ⇒ a $(w, u)$-keyed map is required: *PositionState*.

2. **Shared observables.** `last_settlement_price`, index values, and strategy weights are one per contract and are read identically by every holder (§4.1, §4.3). ⇒ a $u$-keyed map distinct from per-holder state is required: *UnitStatus*.

3. **Two mutation disciplines.** Terms are append-only and versioned for audit and reconstruction; the status cache cell is overwritten on every settle, while the settling event that determines it is appended immutably to the log. Merging the two puts two mutation disciplines under one name (§4.4). ⇒ a third, append-only map is required: *ProductTerms*.

A fourth map (an explicit $W$-sector) adds a sector whose economic content is empty: every candidate datum is $(w, u_{\mathrm{MA}})$-keyed and collapses into *PositionState* (C12, §4.2). Three maps are the minimum basis of the problem, not a compromise.

# 7 Effect on v10.3, Lines 1034 and 2287

v10.3 line 1034 reads:

> *"Unit state is per-unit for most instrument types. For instruments with per-wallet state (e.g., futures accumulated cost), the state dictionary is per (wallet, unit) pair."*

Replaced by:

> Every unit $u \in \mathcal{U}$ carries immutable *ProductTerms*$[u]$ (versioned, append-only) and shared *UnitStatus*$[u]$ (one value per unit, read identically by every holder; a projection of the log, overwritten in place as a read cache, whose every change is caused by a logged event and which replay reconstructs exactly). Every held position $(w, u)$ carries *PositionState*$[w, u]$, with None on the accessor distinguishing "never held" from Some$(0_P)$ "held and flat". State of a holder's relationship to a strategy, mandate, or managed account lives at *PositionState*$[w, u_{\text{MA}}]$, where $u_{\text{MA}}$ is the mandate or strategy unit. There is no separate wallet-keyed state sector.

The migration is additive, then swap: `get_unit_state(u)` remains a deprecated alias for the pair `(product_terms(u), unit_status(u))`; the `get_unit_state(w,u)` of line 2287 maps to `position(w,u)`.

# 8 Alternatives Considered and Rejected

The design is the fixed point of an adversarial multi-agent review (closing note). That review ranked six designs on testability, correctness, and simplicity; the figures below are its ordinal judgments on a 0–10 scale (higher better on all three) — a relative ranking, not measured quantities. Correctness is the gate axis. The per-design forcing reason that follows the table, not the score, carries the argument.

| | **Test.** | **Corr.** | **Simp.** |
|---|---|---|---|
| A — v10.3 current (per-unit, plus per-$(w, u)$ for futures only) | 4 | 5 | 4 |
| **B — three maps + C1–C12 (adopted)** | **9** | **9** | **8** |
| C — partial map with a sentinel unit | 7 | 3 | 7 |
| D — four maps with an explicit $W$-sector | 7 | 7 | 5 |
| E — state indexed only over the held set $\{(w, u) : \texttt{position}(w, u) = \text{Some}\}$ | 8 | 9 | 2 |
| F — two maps with a universe wallet $w_\star$ | 5 | 6 | 6 |

**The one forcing reason each rejected design loses.**

- **A** has no sector for shared observables distinct from immutable terms, so settlement prices and immutable terms share one overwrite-in-place cell — the two mutation disciplines of §6(iii) collapse. Weaker than B on all three axes.
- **C** (sentinel unit) fails day-one correctness: the sentinel has no issuer and no conservation partner, so $\sum_w h(\cdot) = 0$ fails by fiat (§4.2).
- **D** (explicit $W$-sector) adds a sector whose economic content is empty: every candidate datum is $(w, u_{\text{MA}})$-keyed (§6, C12).
- **E** (state indexed only over the held set) equals B on correctness but its construction has no implementation in available tooling, and is radically less simple. Elegance does not decide; shippability does.

- **F** (universe wallet $w_\star$) folds *UnitStatus* into *PositionState*$[w_\star, u]$ and so re-introduces the per-unit / per-$(w, u)$ split as a runtime predicate `is_universe`$(w)$ on the wallet axis. Conservation $\sum_w \Delta\mathtt{ac}(w, u) = 0$ must then exclude $w_\star$ explicitly — a carve-out, not a rule. *UnitStatus* is the type-level expression of what F hides as a wallet-id convention.

B strictly dominates A, C, D, and F on all three axes, and dominates E (equal correctness, higher testability, far higher simplicity). B is the unique Pareto-optimum under any correctness gate $\geq 7$, and is adopted.

# 9 Invariants Made Unrepresentable

Under design B with C1–C12, seven of the ten core invariants of v10.3 §11 are placed beyond reach. The mechanism differs by invariant and is named in each gloss: an abstract type whose only constructor is a checked smart constructor makes the illegal value unable to reach the operation that would act on it (P1); a NonEmptyList makes the versionless state untypable (P6); a per-field tag is a type error at authorship, then erased at the stored row (P10). "Unrepresentable" is used in this precise sense, not as a claim that every guarantee is a type-level fact: conservation, in particular, is a value-level check (§12, signal S4). The invariant statements are those of v10.3 §11; the gloss below names each, its mechanism, and the conditions that discharge it.

**P1** (conservation of quantity) — handler-level per-class zero-sum (C2) and atomic delta (C3). `ValidDelta` is abstract; its only constructor, `validate`, discharges the zero-sum, so an unconserved delta cannot reach `applyDelta`. The check is value-level (signal S4), not a type fact.

**P3** (determinism of replay) — replay is the `foldM` of the event stream, hence a fold homomorphism: `replay (xs <> ys) = replay xs >=> replay ys`, where `f >=> g` is the composition that runs the error-returning step `f`, feeds its result to `g`, and stops at the first error (composition of `Either LedgerError`-returning steps). Replaying a concatenated log equals replaying each part and composing the two. The monotone carrier (C1(b)) keeps the key set stable across any checkpoint cut, so checkpoint-independence is a consequence of this law, not a test.

**P5** (idempotency of lifecycle events) — the lifecycle stage lives in *UnitStatus*$[u]$, $u$-keyed and written by replacement, so re-applying a stage write is the identity (`EXPIRED` over `EXPIRED` is `EXPIRED`): idempotency is structural at a single key, not cross-map coordination, and an untraded unit's lifecycle (§4.4) discharges it touching no *PositionState* row. The non-conserved *PositionState* fields carry the same algebra — `hwm` by max ($\max(x, x) = x$), `entry_nav` write-once — while the additive conserved fields `accumulated_cost` and `balance` draw their replay-safety from P1, not from replacement.

**P6** (immutability of terms) — append-only NonEmptyList[*TermsVersion*] (C6) and registration totality (C7).

**P7** (no in-place mutation of identity) — re-registration rejected (C10); breaking amendments allocate $u_{\text{new}}$ and never rewrite $u_{\text{old}}$ (C8).

**P9** (capability scoping) — cross-$(w, u_{\text{MA}})$ overlay reads forbidden (C4); enforced at the capability layer wrapping the accessors, not in the data reference (signal S2).

**P10** (handler-field canon) — the canonical writer set per field (C11); a write by a field-writer outside it is a type error at authorship, erased once writes share a delta row (signal S3).

No other candidate design makes more than three of the ten unrepresentable.

## 10 Testing Commitments

The specification commits to these figures.

- Event handlers (arithmetic on exact decimals): mutation score **85–90%** — conservation kills sign and coefficient mutants.
- Lifecycle guards (`REGISTERED` → `ACTIVE` → `EXPIRED`): **70–80%** — boundary-comparison mutants (`>` vs `>=`) require targeted tests. The lifecycle is a flat enumeration, so transition ordering (no `EXPIRED` → `REGISTERED`) is enforced by these tests, not by types — distinct from P5, whose idempotency is structural.
- Core overall: **≥80%** (Feathers threshold).
- State-machine model at $|W| = 3$, $|U| = 2$, depth $\leq 6$: $10^5$–$10^6$ reachable states (TLC-tractable); conservation-violating handlers caught at depth 1.

The generator universe (CDM enum × product-type) is finite and enumerable, so coverage is structurally bounded.

## 11 Risk Register

Eight migration, governance, and operational risks. None is a correctness blocker; each is budgeted.

| # | Risk | Mitigation |
|---|---|---|
| F1 | Migration scope: every downstream reader of `unit_state`. | Ship `get_unit_state` as a deprecated alias; split additively before cutover; parallel-run at least one quarter with reconciliation. |
| F2 | Ownership of the C8 fungibility predicate is ungoverned. | Assign per-product RACI in the Unit Store registry; require Legal/Product/Risk sign-off for predicate changes; version the predicate. |
| F3 | Monotone-carrier key-space growth at $10^7$ wallets $\times 10^6$ units is unbenchmarked. | Benchmark Option-accessor cost and cache behaviour before merge; add a tombstone-compaction policy for rows held at Some($0_P$) beyond the retention horizon. |
| F4 | C1–C12 is a multi-sprint programme, not one release. | Land the schema with C1/C5/C6/C7/C9/C10 (type-enforced); stage C2/C3/C8/C11 over later releases; never merge without at least C1, C2, C3. |
| F5 | Mandate-as-unit creates an SFTR / EMIR reporting surface for mandate issuance. | Pre-flight with the Regulatory team; determine whether $u_{\mathrm{MA}}$ issuance requires a UTI / LEI pair; consider a `reportable` flag on $\mathit{ProductTerms}[u_{\mathrm{MA}}]$. |
| F6 | CDM alignment (per-trade `TradeState` vs $\mathit{PositionState}[w, u]$) is asserted, not verified. | Re-run the CDM mapping (Rosetta NS1–7) against the three-map schema before any CDM adapter work; publish a delta document. |

| | | |
|---|---|---|
| F7 | Onboarding cost of three maps, Option/monotone, and twelve conditions is real. | Ship a decision tree and runbook; require training before handler work; provide a one-page cheat sheet. |
| F8 | Forward migration is irreversible once the $W$-sector is collapsed. | Keep a read-only mirror of the prior wallet-state for at least four quarters post-cutover; document the inverse mapping (overlay key $u_{\mathrm{MA}} \to$ former field) explicitly. |

F2, F5, and F6 require external stakeholders before any code ships. F1, F3, F4, F7, and F8 are internal engineering risks, manageable within the delivery programme.

## 12 Reference Implementation

The reference is `reference/StatesHome.hs`, included below. It is total and deterministic, and makes the illegal states unrepresentable rather than merely checked. Quantities are exact integer minor units, not floating point, so determinism and conservation are arithmetic facts.

The encoding carries the conditions structurally:

- `Ledger` is abstract with no row deleter, so no caller can remove a *PositionState* row — the monotone half of C1(b). `position` returns `Maybe` — the Option half of C1(a).
- `ValidDelta` is abstract; its only constructor is `validate`, which discharges conservation as a monoid identity (C2). An unconserved delta cannot reach `applyDelta`. The empty fold is `mempty`, so zero-holder events validate with no special case (C9).
- `ProductTerms` is abstract over a `NonEmpty`; the only growth operations are `register` (singleton, C7) and `appendVersion` (append, C6). No setter exists.
- The `FieldWrite h` GADT constructors *are* the C11 field-to-writer table: a write by the wrong handler fails to typecheck at the handler's authorship site. `settleHandler` has output type `Map WalletId [FieldWrite 'Settle]` and is the live call site that exercises this: `main` builds `tradeSD` and `closeSD` as `erase . settleHandler`, erasing the index through `erase = fmap (map SomeWrite)` as the writes enter `sdRows`, so the guarantee binds at authorship, not at the delta row (signal S3). The commented `_c11_bad` witnesses the rejected cross-handler write.
- `applyDelta` returns the whole new ledger or a `Left`; there is no partial state (C3). `register` rejects re-registration with `ReRegistration` (C10) and writes *ProductTerms* and *UnitStatus* together, so "registered in PT $\iff$ registered in US" holds by construction.
- `amend` is the two-track C8: `Preserving` appends; `Breaking` allocates a fresh unit and stamps `usSupersededBy`, never rewriting the old terms.

```
{-# LANGUAGE GADTs              #-}
{-# LANGUAGE DataKinds          #-}
{-# LANGUAGE KindSignatures     #-}
{-# LANGUAGE StandaloneDeriving #-}

-- ============================================================================
-- StatesHome.hs  --  reference for Addendum A1 (StatesHome) of The Ledger v10.3.
--
-- Replaces the Python listing of the addendum's "Minimal Reference
-- Implementation". Goal: make the addendum's illegal states UNREPRESENTABLE and
```

```
-- keep every exported function TOTAL and DETERMINISTIC.
--
-- The export list below is part of the specification, not decoration:
--   * `Ledger` is exported ABSTRACT (no field setters) -- the only way to change
--     PositionState is `applyDelta`, which inserts/updates rows and NEVER deletes
--     one. There is deliberately no PS deleter anywhere. That absence IS the
--     monotone-carrier discipline (C1, storage half).
--   * `ValidDelta` is exported ABSTRACT -- the only way to build one is `validate
    `,
--     which discharges conservation (C2). An unconserved delta cannot reach
--     `applyDelta`. The illegal state "applied an unconserved delta" is
    unreachable.
--   * `ProductTerms` is exported ABSTRACT -- the only ways to grow it are
--     `register` (singleton) and `appendVersion` (append). There is no in-place
--     setter. That absence IS the append-only discipline (C6).
--
-- A note on numbers: quantities are exact `Integer` minor units, never `Float`
-- (the Python listing used `float`). Determinism and conservation are arithmetic
-- facts; floating point would forfeit both. This is the single deviation from the
-- Python and it is a correctness improvement, not a translation choice.
-- ============================================================================

module StatesHome
  ( -- * Scalars (exact, an additive abelian group)
    Qty (..), qneg, qmax
    -- * Keys
  , WalletId (..), UnitId (..)
    -- * Map 1: ProductTerms  (immutable, versioned, append-only -- C6/C7)
  , TermsVersion (..)
  , ProductTerms            -- abstract: no in-place setter is exported (C6)
  , currentTerms, allVersions, appendVersion
    -- * Map 2: UnitStatus  (projection of the log, overwritten in place as a
    --   read cache; shared, read identically by every holder -- C5)
  , Lifecycle (..), UnitStatus (..), defaultStatus
    -- C11 (UnitStatus half): the closed status-writer set. `applyStatus` is the
    -- ONLY function that writes a UnitStatus field; every change is a case of it.
  , StatusWrite (..), applyStatus
    -- * Map 3: PositionState (per (w,u); Option accessor + monotone carrier -- C1
    )
  , PositionState (..), zeroP
    -- * C11: per-field canonical-writer tagging (a write names its handler in its
     type)
  , Handler (..), FieldWrite (..), SomeWrite (..), applyWrite, conserved
  , settleHandler, erase     -- the live authorship -> erasure pipeline (C11/S3)
    -- * Conservation (C2) as a group homomorphism into PosDelta
  , PosDelta (..)
    -- * Atomic, conservation-checked event delta (C2/C3)
  , StateDelta (..), ValidDelta, validate, ConservationError (..)
    -- * The ledger and its total operations
  , Ledger                  -- abstract: no row-deleter is exported (C1, monotone)
  , emptyLedger, register, applyDelta, replay
  , productTerms, unitStatus, position
  , LedgerError (..)
    -- * C8: two-track amendment
  , Fungibility (..), FungibilityPredicate, AmendResult (..), amend
    -- * Runnable example
  , main
  ) where
```

```
import           Control.Monad      (foldM)
import           Data.List          (foldl')
import           Data.List.NonEmpty (NonEmpty (..))
import qualified Data.List.NonEmpty as NE
import           Data.Map.Strict    (Map)
import qualified Data.Map.Strict    as Map

-- ============================================================================
-- Scalars.  Qty is the additive abelian group of exact minor units.
-- Conservation (C2) is a statement in THIS monoid: a conserving delta sums to
-- `mempty`. Using a monoid here is not ornament -- it is what lets the
-- zero-holder (vacuous) base case fall out for free: a sum over no wallets is
-- `mempty`, i.e. conserved, with no special case (C9).
-- ============================================================================

newtype Qty = Qty Integer deriving (Eq, Ord)
instance Show Qty where show (Qty n) = show n

instance Semigroup Qty where Qty a <> Qty b = Qty (a + b)
instance Monoid    Qty where mempty = Qty 0

qneg :: Qty -> Qty
qneg (Qty n) = Qty (negate n)

qmax :: Qty -> Qty -> Qty
qmax (Qty a) (Qty b) = Qty (max a b)

-- Keys.
newtype WalletId = WalletId String deriving (Eq, Ord, Show)
newtype UnitId   = UnitId   String deriving (Eq, Ord, Show)

-- ============================================================================
-- Map 1 -- ProductTerms : Map UnitId (NonEmpty TermsVersion)
--   immutable, versioned, APPEND-ONLY (C6), registration-total (C7).
--
-- `NonEmpty` makes "registered but versionless" unrepresentable: there is no
-- ProductTerms with zero versions, so `currentTerms` is total without a Maybe.
-- The type is abstract; the only growth operations are `register` (which builds
-- the singleton) and `appendVersion`. No setter exists -> C6 holds by absence.
-- ============================================================================

data TermsVersion = TermsVersion
  { tvLabel  :: String
  , tvFields :: Map String String   -- opaque terms payload (multiplier, ISIN,
    ...)
  } deriving (Eq, Show)

newtype ProductTerms = ProductTerms (NonEmpty TermsVersion) deriving (Show)

-- | The version in force = the most recently appended one. Total (NonEmpty).
currentTerms :: ProductTerms -> TermsVersion
currentTerms (ProductTerms vs) = NE.last vs

allVersions :: ProductTerms -> NonEmpty TermsVersion
allVersions (ProductTerms vs) = vs

-- | C6: the ONLY in-module mutation of terms. Append; never rewrite.
```

```
appendVersion :: TermsVersion -> ProductTerms -> ProductTerms
appendVersion tv (ProductTerms vs) = ProductTerms (vs <> (tv :| []))

-- ============================================================================
-- Map 2 -- UnitStatus : Map UnitId UnitStatus
--   A materialised projection of the log -- a cached view the log can always
--   rebuild (C5).
--
-- UnitStatus holds one value per unit, shared -- read identically by every
-- holder. Its value changes over time, but UnitStatus is not a separate source
-- of truth: the immutable event log is. Every change is caused by a logged
-- event; the stored value is overwritten in place only as a read cache.
-- Replaying the events up to any point rebuilds the exact value that held then,
-- so nothing is lost by overwriting and there is no other way to change the
-- value. A value exists from registration onward.
--
-- Materialisation-soundness (held here by construction): the stored value always
-- equals what replaying the unit's events produces, and no part of the system
-- writes it except through a logged event. The single writer is `applyStatus`
-- (below): every UnitStatus field changes ONLY as a case of that pure total
-- step, folded over the unit's status events inside `applyDelta`. No exported
-- `setStatus` exists, no record-update on a UnitStatus value appears anywhere
-- else in the module, and the `Ledger` is sealed -- so no out-of-band writer can
-- reach a field, and the cache cannot drift from the log.
--
-- This is the shared-observable sector: lifecycle, `last_settlement_price`,
-- `superseded_by`. Because every holder reads the same value, it is shared
-- environment, not per-holder state.
--
-- Boundary note: an externally-sourced number (a settlement price, a benchmark
-- level) enters ONLY as a logged observation event carrying the snapshotted
-- value, so the fold that rebuilds the status stays pure -- a fresh replay
-- reproduces the same number bit for bit. (Reproducing the number is assured
-- here; ATTESTATION that the number is the genuine external level is a distinct
-- requirement, recorded separately, not addressed by this storage discipline.)
-- ============================================================================

data Lifecycle = Listed | Active | Expired | Closed deriving (Eq, Show)

data UnitStatus = UnitStatus
  { usLifecycle    :: Lifecycle
  , usLastSettle   :: Maybe Qty       -- None at registration default (C5)
  , usSupersededBy :: Maybe UnitId    -- set by a Breaking amendment (C8)
  } deriving (Eq, Show)

-- | C5: product-declared default, applied at registration. `LISTED`, no settle
--   price, not superseded. Because `register` always writes this, `unitStatus`
--   is total on every registered unit.
defaultStatus :: UnitStatus
defaultStatus = UnitStatus Listed Nothing Nothing

-- ============================================================================
-- C11 (UnitStatus half) -- the closed set of UnitStatus field writers.
--
-- Each constructor is the ONE canonical writer of its field; `applyStatus` is
-- the ONLY function in the module that writes a UnitStatus field. No
-- record-update on a UnitStatus value appears anywhere else -- so a field cannot
-- change except as a case of this pure total step. `applyDelta` folds the
```

```haskell
-- StatusWrites carried by an event over the unit's stored status; this is the
-- "rule for folding each kind of event" that keeps the stored value equal to
-- the replay of the log: overwrite-in-place, last-write-wins (every write is
-- idempotent, P5), never an off-log patch.
--
-- Genesis is separate and singular: `register` sets the initial value once, from
-- the product-declared `defaultStatus`. Thereafter every change is a StatusWrite.
-- ============================================================================

data StatusWrite
  = SetLifecycle    Lifecycle  -- lifecycle event: Listed -> Active -> Expired ->
    Closed
  | SetLastSettle   Qty        -- settle event: the marked settlement price (a
    logged observation)
  | SetSupersededBy UnitId     -- Breaking amendment (C8): point the old unit at
    its successor
  deriving (Eq, Show)

-- | Apply one status write to a unit's status. Total over all constructors;
--   each case overwrites exactly one field (last-write-wins, idempotent).
applyStatus :: StatusWrite -> UnitStatus -> UnitStatus
applyStatus (SetLifecycle l)    s = s { usLifecycle    = l }
applyStatus (SetLastSettle q)   s = s { usLastSettle   = Just q }
applyStatus (SetSupersededBy u) s = s { usSupersededBy = Just u }

-- ============================================================================
-- Map 3 -- PositionState : Map (WalletId, UnitId) PositionState
--   per (holder, unit). C1 has TWO orthogonal halves, both required:
--     (a) Option accessor  : `position` returns `Maybe PositionState`.
--           Nothing      = this wallet has NEVER held this unit.
--           Just zeroP   = held once, currently flat.
--         These cannot be collapsed (VM-settle, wash-sale lookback, record-date
--         entitlements all read the difference).
--     (b) Monotone carrier : a row, once created, is never deleted. Close-out
--         leaves a flat row. Enforced structurally: `applyDelta` only
--         inserts/updates, and no deleter is exported.
--
-- Field disciplines, demonstrating C11 (one canonical writer per field):
--   psAc       conserved, additive            <- Settle / Trade
--   psBalance  conserved, additive            <- Transfer
--   psHwm      NOT conserved, monotone (max)  <- FeeCrystallise
--   psEntryNav NOT conserved, write-once      <- Subscribe
-- Note: "held-and-flat" means psAc = psBalance = 0; psHwm / psEntryNav are
-- retained (final HWM kept for tax reporting -- addendum wind-down case).
-- ============================================================================

data PositionState = PositionState
  { psAc       :: !Qty
  , psBalance  :: !Qty
  , psHwm      :: !Qty
  , psEntryNav :: !(Maybe Qty)
  } deriving (Eq, Show)

-- | The flat row created on first touch (also the "held-and-flat" baseline).
zeroP :: PositionState
zeroP = PositionState mempty mempty mempty Nothing

-- ============================================================================
```

```
-- C11 -- per-field canonical-writer-set tagging, as a TYPE-LEVEL relation.
--
-- A 'FieldWrite h' is a write performed BY handler 'h'. The GADT constructors
-- ARE the C11 field->writer table; no other pairing is representable:
--     ac       writable by Settle and Trade   (two constructors)
--     balance  writable by Transfer
--     hwm      writable by FeeCrystallise
--     entryNav writable by Subscribe
-- Each event handler is typed by the Handler it speaks for. 'settleHandler'
-- (defined below, and the source of 'main''s trade/close deltas) has output type
--     settleHandler :: ... -> Map WalletId [FieldWrite 'Settle]
-- so the phantom index constrains at a LIVE call site, not only in an alias. A
-- settle handler that tried to bump hwm would have to produce 'WHwm', whose type
-- is 'FieldWrite 'FeeCrystallise', and would fail to typecheck against the
-- declared ''Settle' output. C11's "mutation by any other handler is a type
-- error" is therefore literally a type error -- exercised by 'settleHandler',
-- with the static witnesses '_c11_ok_*' and the commented '_c11_bad' below.
-- ============================================================================

data Handler = Settle | Trade | Transfer | FeeCrystallise | Subscribe
  deriving (Eq, Show)

data FieldWrite (h :: Handler) where
  WAc       :: Qty -> FieldWrite 'Settle           -- ac, by settle
  WAcTrade  :: Qty -> FieldWrite 'Trade            -- ac, by trade
  WBalance  :: Qty -> FieldWrite 'Transfer         -- balance, by transfer
  WHwm      :: Qty -> FieldWrite 'FeeCrystallise   -- hwm, by fee crystallisation
  WEntryNav :: Qty -> FieldWrite 'Subscribe        -- entry_nav, by subscribe

deriving instance Show (FieldWrite h)

-- | Handler index erased for storage in a heterogeneous delta row. The index
--   was already checked at the point each handler authored its write (see the
--   EXPRESSIBILITY SIGNAL on C11 below).
data SomeWrite where
  SomeWrite :: FieldWrite h -> SomeWrite

instance Show SomeWrite where show (SomeWrite w) = show w

-- | Apply one field write to a row. Total over all constructors.
applyWrite :: FieldWrite h -> PositionState -> PositionState
applyWrite (WAc q)       p = p { psAc      = psAc p <> q }
applyWrite (WAcTrade q)  p = p { psAc      = psAc p <> q }
applyWrite (WBalance q)  p = p { psBalance = psBalance p <> q }
applyWrite (WHwm q)      p = p { psHwm     = qmax (psHwm p) q }                -- monotone
applyWrite (WEntryNav q) p = p { psEntryNav = maybe (Just q) Just (psEntryNav p) }
    -- write-once

-- | A settle handler authors ONLY ''Settle' writes. Its result type
--   'Map WalletId [FieldWrite 'Settle]' IS the C11 checkpoint: this is the live
--   call site at which the phantom index constrains. A body that tried to bump
--   hwm would have to emit 'WHwm :: FieldWrite 'FeeCrystallise' and could not
--   typecheck against this signature. 'main' builds its trade/close deltas from
--   this handler, so C11 is exercised on the running path, not only in an alias.
settleHandler :: [(WalletId, Qty)] -> Map WalletId [FieldWrite 'Settle]
settleHandler legs = Map.fromList [ (w, [WAc q]) | (w, q) <- legs ]
```

```haskell
-- | The S3 boundary, in code. Erase the per-handler index so heterogeneous
--   authored writes can share one delta row; authorship was already typechecked
--   at the handler's output type. After `erase` the row is `[SomeWrite]` and the
--   index is gone. This is the authorship -> erasure pipeline the addendum names.
erase :: Map WalletId [FieldWrite h] -> Map WalletId [SomeWrite]
erase = fmap (map SomeWrite)

-- ============================================================================
-- Conservation (C2) as a group homomorphism.
--
-- `conserved` maps each field write to its contribution in the abelian group
-- `PosDelta` of conserved-field deltas (ac, balance). Non-conserved fields
-- (hwm, entryNav) contribute the identity.
--
-- Totalling a delta over wallets is then a monoid homomorphism
--     Map WalletId [SomeWrite]  -->  PosDelta
-- and "conserving" means the image is `mempty`. State the law in words:
--     for every event class, the conserved fields sum to zero across wallets.
-- The categorical name is the LAST thing said, not the first: it is a group
-- homomorphism landing at the identity.
--
-- Per-event-class cases, all the same identity in `PosDelta`:
--   2-leg trade : WAc(+q) <> WAc(-q)            = mempty
--   K-leg       : <> of K balanced legs          = mempty
--   VM fan-out  : each wallet reset, cash offsets = mempty
--   vacuous     : <> over the EMPTY map          = mempty   (C9; no holders, no
--                 division by holder count -- the dividend/len(holders) bug
--                 class cannot arise because we sum deltas, never divide).
-- ============================================================================

data PosDelta = PosDelta { dAc :: !Qty, dBalance :: !Qty } deriving (Eq, Show)

instance Semigroup PosDelta where
  PosDelta a b <> PosDelta a' b' = PosDelta (a <> a') (b <> b')
instance Monoid PosDelta where
  mempty = PosDelta mempty mempty

conserved :: FieldWrite h -> PosDelta
conserved (WAc q)       = PosDelta q mempty
conserved (WAcTrade q)  = PosDelta q mempty
conserved (WBalance q)  = PosDelta mempty q
conserved (WHwm _)      = mempty
conserved (WEntryNav _) = mempty

-- ============================================================================
-- Atomic, conservation-checked StateDelta (C2 + C3).
--
-- A StateDelta is the proposed change for ONE event, across all three maps:
--   sdRows   : PositionState changes, keyed by wallet (the unit is sdUnit).
--   sdStatus : the UnitStatus field writes this event causes (each a case of
--              `applyStatus`); empty if the event touches no status field. The
--              event log carries these writes, so replay rebuilds the status.
--   sdAppend : a TermsVersion to append, if the event touches terms.
-- C3 (atomic, all-or-nothing) is structural: `applyDelta` returns the whole new
-- `Ledger` or a `Left`. There is no observable partially-applied state -- a
-- rejected delta (unconserved, caught by `validate`; or on an unregistered unit,
-- caught by `applyDelta`) leaves the prior ledger untouched.
-- ============================================================================
```

```
data StateDelta = StateDelta
  { sdUnit   :: UnitId
  , sdRows   :: Map WalletId [SomeWrite]
  , sdStatus :: [StatusWrite]
  , sdAppend :: Maybe TermsVersion
  } deriving (Show)

-- | C2 witness. Abstract: the ONLY constructor is `validate` (Show is for the
--   example's benefit; the data constructor is not exported).
newtype ValidDelta = ValidDelta StateDelta deriving (Show)

data ConservationError = NotConserved UnitId PosDelta deriving (Show)

-- | Discharge conservation. The conserved fold over the rows must be the
--   identity. `foldMap` over the `Map` folds its values; over `[SomeWrite]`
--   folds the list; the empty case is `mempty` (C9), so zero-holder events
--   validate with no special case.
validate :: StateDelta -> Either ConservationError ValidDelta
validate sd
  | net == mempty = Right (ValidDelta sd)
  | otherwise     = Left  (NotConserved (sdUnit sd) net)
  where
    net :: PosDelta
    net = foldMap (foldMap (\(SomeWrite w) -> conserved w)) (sdRows sd)

-- ============================================================================
-- The ledger.  Abstract: no field setter is exported, so no caller can delete a
-- PositionState row (monotone carrier, C1) or fabricate terms (C6) / a valid
-- delta (C2) from outside.
-- ============================================================================

data Ledger = Ledger
  { ledgerPT :: Map UnitId ProductTerms
  , ledgerUS :: Map UnitId UnitStatus
  , ledgerPS :: Map (WalletId, UnitId) PositionState
  }

-- | For the runnable example only (e.g. `print (applyDelta ...)`); `Ledger`
--   remains ABSTRACT through the export list, so this grants no field access.
deriving instance Show Ledger

data LedgerError
  = ReRegistration   UnitId   -- C10
  | UnitAlreadyExists UnitId   -- fresh-id collision in a Breaking amendment
  | UnknownUnit      UnitId
  deriving (Eq, Show)

emptyLedger :: Ledger
emptyLedger = Ledger Map.empty Map.empty Map.empty

-- | C5 + C7 jointly, by construction: `register` is the ONLY introducer of a
--   unit, and it writes BOTH ProductTerms and UnitStatus, establishing the
--   invariant "u registered in PT  <=>  u registered in US". Every other mutation
--   path PRESERVES it: `applyDelta` touches US/PT only for an already-registered
--   `sdUnit` (it rejects an unregistered one with `UnknownUnit`), and `amend`'s
--   Breaking track writes PT and US for the fresh unit together. The invariant is
--   thus preserved by construction across the whole module, not merely asserted.
```

```
--   C10: re-registration is a hard `Left`, never a silent reset.
register :: UnitId -> TermsVersion -> UnitStatus -> Ledger -> Either LedgerError
    Ledger
register u tv us l
  | u `Map.member` ledgerPT l = Left (ReRegistration u)        -- C10
  | otherwise = Right l
      { ledgerPT = Map.insert u (ProductTerms (tv :| [])) (ledgerPT l)  -- C7
      , ledgerUS = Map.insert u us                          (ledgerUS l)  -- C5
      }

-- | Apply a validated event to a ledger. Two failure modes, both typed:
--   conservation (already discharged by `validate`, so it cannot recur here) and
--   REGISTRATION -- a delta whose `sdUnit` is not registered is a hard
--   `Left (UnknownUnit u)`. The registration guard is what makes the PT<->US
--   invariant hold BY CONSTRUCTION rather than by assertion: because the guard
--   establishes `u` is in `ledgerPT` (the canonical registration test -- see
--   `register`), the UnitStatus write can only REPLACE an existing entry (it
--   cannot fabricate a half-registered unit), and the terms `Map.adjust` is
--   guaranteed to hit (the append can no longer vanish silently). Conservation
--   and registration are deliberately separated: `validate` is ledger-
--   independent (a pure conservation witness, valid in any ledger), so the
--   contextual registration check belongs here, at apply time, not baked into a
--   would-be-stale `ValidDelta`.
--   Atomic: returns the whole new ledger or a `Left`, never a partial state.
--   Monotone: rows are inserted/updated, never removed.
applyDelta :: ValidDelta -> Ledger -> Either LedgerError Ledger
applyDelta (ValidDelta sd) l
  | not (u `Map.member` ledgerPT l) = Left (UnknownUnit u)
  | otherwise = Right l
      { ledgerPS = Map.foldrWithKey applyRow (ledgerPS l) (sdRows sd)
      , ledgerUS = if null (sdStatus sd)
                   then ledgerUS l
                   else Map.adjust (\s -> foldl' (flip applyStatus) s (sdStatus
    sd)) u (ledgerUS l)
                        -- overwrite-in-place as a read cache, one field per
    StatusWrite;
                        -- u is registered, so adjust HITS and can only REPLACE
    field values --
                        -- it cannot fabricate a half-registered unit, and there
     is no other
                        -- door to a UnitStatus field than this fold of `
    applyStatus`.
      , ledgerPT = maybe (ledgerPT l)
                         (\tv -> Map.adjust (appendVersion tv) u (ledgerPT l))  --
     C6 append; hits (u registered)
                         (sdAppend sd)
      }
  where
    u = sdUnit sd
    applyRow w writes ps =
      let key = (w, u)
          cur = Map.findWithDefault zeroP key ps                  -- first touch
    -> flat row
          new = foldl' (\acc (SomeWrite fw) -> applyWrite fw acc) cur writes
      in  Map.insert key new ps                                   -- insert/update
     only

-- | Replay = an error-stopping fold of the event stream over the ledger.
```

```
--   Determinism of replay (P3) is the fold-homomorphism law: for event streams
--   xs, ys,
--       replay (xs <> ys)  =  replay xs >=> replay ys
--   i.e. replaying two batches of events in sequence gives the same result as
--   replaying them together, stopping at the first error, where `f >=> g` runs
--   the error-returning step `f`, feeds its result to `g`, and stops at the first
--   error (composition of `Either LedgerError`-returning steps). Hence `replay
--   events` is independent of where a checkpoint is cut -- checkpoint-
--   independence is a CONSEQUENCE of the law, not a test. The monotone carrier
--   (C1) keeps the key set stable across cuts; on a well-formed stream (every
--   delta on a registered unit) no `Left` arises, so replay is total there.
replay :: [ValidDelta] -> Ledger -> Either LedgerError Ledger
replay ds l0 = foldM (\l d -> applyDelta d l) l0 ds

-- Accessors. Total. `Maybe` on PT/US distinguishes unregistered; on PS it is
-- the load-bearing C1 Option accessor (never-held vs held-and-flat).
productTerms :: Ledger -> UnitId -> Maybe ProductTerms
productTerms l u = Map.lookup u (ledgerPT l)

unitStatus :: Ledger -> UnitId -> Maybe UnitStatus
unitStatus l u = Map.lookup u (ledgerUS l)

position :: Ledger -> WalletId -> UnitId -> Maybe PositionState   -- C1 Option
    accessor
position l w u = Map.lookup (w, u) (ledgerPS l)

-- ============================================================================
-- C8 -- two-track amendment.
--
-- A product-declared, total predicate decides the track:
--     is_fungibility_preserving : ProductTerms -> TermsVersion -> Fungibility
--   Preserving -> append a TermsVersion to the SAME unit (C6); existing
--                 PositionState rows survive untouched.
--   Breaking   -> allocate a FRESH unit; stamp `superseded_by` on the old
--                 unit's status; the old unit's terms are NEVER rewritten
--                 (C7/P7). Re-subscription that moves holders old -> new is a
--                 separate paired-issuance event -- see the EXPRESSIBILITY
--                 SIGNAL on cross-unit conservation below.
-- ============================================================================

data Fungibility = Preserving | Breaking deriving (Eq, Show)

type FungibilityPredicate = ProductTerms -> TermsVersion -> Fungibility

data AmendResult
  = Appended   UnitId          -- Preserving: same unit, new version
  | Superseded UnitId UnitId   -- Breaking: old unit -> fresh unit
  deriving (Eq, Show)

amend :: FungibilityPredicate
      -> UnitId        -- ^ unit being amended
      -> TermsVersion  -- ^ the amendment
      -> UnitId        -- ^ caller-supplied fresh id (used only on the Breaking
    track)
      -> Ledger
      -> Either LedgerError (AmendResult, Ledger)
amend isFungible u tvNew uFresh l =
  case Map.lookup u (ledgerPT l) of
```

```
    Nothing -> Left (UnknownUnit u)
    Just pt -> case isFungible pt tvNew of
      Preserving ->
        Right ( Appended u
              , l { ledgerPT = Map.insert u (appendVersion tvNew pt) (ledgerPT l)
    } )
      Breaking
        | uFresh `Map.member` ledgerPT l -> Left (UnitAlreadyExists uFresh)
        | otherwise ->
            Right ( Superseded u uFresh
                  , l { ledgerPT = Map.insert uFresh (ProductTerms (tvNew :| [])) 
    (ledgerPT l)
                      , ledgerUS = Map.insert uFresh defaultStatus
                                 $ Map.adjust (applyStatus (SetSupersededBy uFresh
    )) u
                                              (ledgerUS l)
                      } )

-- | C11, made concrete. These typecheck:
_c11_ok_settle :: Qty -> FieldWrite 'Settle
_c11_ok_settle = WAc
_c11_ok_fee    :: Qty -> FieldWrite 'FeeCrystallise
_c11_ok_fee    = WHwm
-- ...and this would NOT (uncomment to see the C11 type error):
--   _c11_bad :: Qty -> FieldWrite 'Settle
--   _c11_bad = WHwm        -- WHwm :: FieldWrite 'FeeCrystallise, not 'Settle

-- ============================================================================
-- EXPRESSIBILITY SIGNALS  (recorded, not contorted around)
-- ----------------------------------------------------------------------------
-- [S1] Cross-unit conservation for the Breaking-amendment re-subscription.
--      POINTS AT: the DESIGN / NOTATION (not this encoding).
--      Conservation is stated per unit: for each u, Sum_w df(w,u) = 0. A
--      re-subscription "moves" (w, u_old) -> (w, u_new); per unit that is NOT
--      zero-sum (one wallet gains on u_new with no offsetting wallet on u_new).
--      A single-unit StateDelta cannot express it. The faithful encoding is
--      PAIRED ISSUANCE: a burn on u_old (client -q, issuer +q ; sums to 0 on
--      u_old) and a mint on u_new (issuer -q, client +q ; sums to 0 on u_new),
--      applied atomically as two ValidDeltas. This is not a contortion -- it
--      surfaces exactly the addendum's own point that u_MA has a REAL issuer
--      (not the Dirac u_empty hack). The signal: state conservation as a
--      homomorphism INDEXED BY UNIT, and model re-subscription as paired
--      issuance, so `applyAll [burn, mint]` is the atomic re-subscription.
--
-- [S2] C4 (capability-scoped reads; cross-(w,u_MA) overlay reads forbidden) is
--      NOT expressed here. POINTS AT: the DESIGN -- read scoping is an
--      authority/effect concern at the boundary (a Reader of capabilities over
--      the accessors), not a shape of the stored data. Forcing it into this
--      pure data reference would buy nothing and add noise. It belongs in the
--      capability layer that wraps `position` / `unitStatus`.
--
-- [S3] C11's type-level guarantee holds at AUTHORSHIP, then is erased.
--      POINTS AT: a DESIGN TRADEOFF (restraint rule). The handler index on
--      `FieldWrite h` is checked where each handler declares its output type:
--      `settleHandler :: ... -> Map WalletId [FieldWrite 'Settle]`. `erase`
--      (= `fmap (map SomeWrite)`) is the boundary -- once heterogeneous writes
--      share a delta row they are wrapped in `SomeWrite` and the index is gone.
--      `main` builds `tradeSD`/`closeSD` as `erase . settleHandler`, so the
```

```
--      authorship -> erasure pipeline runs on the live path, not only in the
--      static `_c11_ok_*` witnesses. Re-proving canon at the row level would need
--      an indexed/heterogeneous list and buys nothing the authorship-site check
--      did not already buy. We stop at the cheaper guarantee on purpose.
--
-- [S4] Conservation cannot be a pure TYPE fact (the addendum's own C2 point:
--      refinement types on decimal sums are not free). We use a value-level
--      smart constructor (`validate` -> `ValidDelta`) returning `Either`. This
--      is the correct boundary, acknowledged, not awkward: types make the
--      *unchecked* delta unable to reach `applyDelta`; the *check itself* is a
--      one-line monoid identity.
--
-- [S5] UnitStatus single-writer closure uses a PLAIN closed sum (`StatusWrite`),
--      NOT the phantom-`Handler` index that `FieldWrite` carries.
--      POINTS AT: a DESIGN TRADEOFF (restraint rule). The hazard the verdict
--      names is OUT-OF-BAND mutation -- a field patched without a logged event.
--      A closed sum whose every constructor names one field, applied only by the
--      total `applyStatus`, with no record-update elsewhere and the `Ledger`
--      sealed, closes exactly that hazard: a UnitStatus field cannot change
--      except as a case of `applyStatus` folded over the log, so the stored cell
--      stays equal to the replay. The `FieldWrite` phantom index additionally
--      ties a write to a HANDLER (a settle handler cannot author a hwm write);
--      UnitStatus has no handler-typing layer here for such an index to
--      constrain, so adding it would buy nothing and add noise. We stop at the
--      per-field closed writer set on purpose -- the fewest primitives that shut
--      the door named by the synthesis.
-- ============================================================================

-- ============================================================================
-- Runnable example.  `runghc StatesHome.hs`  (or load in GHCi and run `main`).
-- The `expect*` helpers are demo glue and are the only partial code in the file;
-- the library above is total. They fail loudly only on an outcome the example
-- asserts cannot happen.
-- ============================================================================

expect :: Show e => Either e a -> IO a
expect = either (\e -> error ("unexpected Left: " <> show e)) pure

main :: IO ()
main = do
  let uES    = UnitId "CME-ES"      -- a future (CME-ES and ICE-ES are DISTINCT
    units)
      uMA    = UnitId "MANDATE-7"   -- a managed-account mandate, itself a unit
      wBuyer = WalletId "BUYER"
      wSeller= WalletId "SELLER"

  -- Registration (C5 + C7): PT and US written together.
  l0 <- expect $ register uES (TermsVersion "ES-v1"
                   (Map.fromList [("multiplier","50"),("ccy","USD")]))
    defaultStatus emptyLedger
  l1 <- expect $ register uMA (TermsVersion "MA-v1"
                   (Map.fromList [("fee","0.02")])) defaultStatus l0

  putStrLn "== untraded instrument (C1 None; US total) =="
  print (position l1 wBuyer uES)            -- Nothing : never held
  print (unitStatus l1 uES)                 -- Just (UnitStatus Listed Nothing
    Nothing)
```

```
putStrLn "== conserving Trade (C2/C3): buyer +1000, seller -1000 =="
let tradeSD = StateDelta uES
      -- C11 authored at type 'Map WalletId [FieldWrite 'Settle]', then S3-
  erased:
      (erase (settleHandler [ (wBuyer , Qty   1000 )
                            , (wSeller, Qty (-1000)) ]))
      [] Nothing
vTrade <- expect (validate tradeSD)
l2 <- expect (applyDelta vTrade l1)
print (position l2 wBuyer  uES)           -- Just (ac=1000)
print (position l2 wSeller uES)           -- Just (ac=-1000)

putStrLn "== non-conserving delta is REJECTED (cannot reach applyDelta) =="
let badSD = StateDelta uES
      (Map.fromList [(wBuyer, [SomeWrite (WAc (Qty 1000))])]) [] Nothing
print (validate badSD)                    -- Left (NotConserved CME-ES (PosDelta
   1000 0))

putStrLn "== zero-holder lifecycle event validates vacuously (C9) =="
let lifeSD = StateDelta uMA Map.empty [SetLifecycle Active] Nothing
vLife <- expect (validate lifeSD)         -- empty fold = mempty : conserved
l3 <- expect (applyDelta vLife l2)
print (usLifecycle <$> unitStatus l3 uMA) -- Just Active ; no PS rows created
print (position l3 (WalletId "ANY") uMA)  -- Nothing

putStrLn "== close-out keeps a flat row (monotone carrier; held-and-flat /=
  never-held) =="
let closeSD = StateDelta uES
      -- same C11 settle authorship, S3-erased into the delta row:
      (erase (settleHandler [ (wBuyer , Qty (-1000))
                            , (wSeller, Qty   1000 ) ]))
      [] Nothing
vClose <- expect (validate closeSD)
l4 <- expect (applyDelta vClose l3)
print (position l4 wBuyer uES)            -- Just (ac=0) : retained, NOT Nothing

putStrLn "== amendment, Preserving track (C8/C6): append a version =="
(r1, l5) <- expect $ amend feePreserving uES
              (TermsVersion "ES-v2" (Map.fromList [("multiplier","50"),("ccy","
  USD")]))
              (UnitId "unused") l4
print r1                                  -- Appended CME-ES
print (tvLabel . currentTerms <$> productTerms l5 uES)  -- Just "ES-v2"
print (length . NE.toList . allVersions <$> productTerms l5 uES)  -- Just 2

putStrLn "== amendment, Breaking track (C8): fresh unit + SupersededBy =="
(r2, l6) <- expect $ amend isinBreaking uMA
              (TermsVersion "MA-v2-newISIN" Map.empty) (UnitId "MANDATE-7b") l5
print r2                                  -- Superseded MANDATE-7 MANDATE-7b
print (usSupersededBy <$> unitStatus l6 uMA)  -- Just (Just (UnitId "MANDATE-7b
  "))
print (tvLabel . currentTerms <$> productTerms l6 (UnitId "MANDATE-7b"))  --
  Just "MA-v2-newISIN"

putStrLn "== delta on an UNREGISTERED unit is REJECTED (PT<->US invariant
  preserved) =="
let ghostSD = StateDelta (UnitId "GHOST") Map.empty [SetLifecycle Listed]
  Nothing
```

```
  vGhost <- expect (validate ghostSD)       -- validates vacuously (empty fold =
    mempty)
  print (applyDelta vGhost l6)              -- Left (UnknownUnit (UnitId "GHOST"))
     : no half-registered unit
  print (unitStatus l6 (UnitId "GHOST"))    -- Nothing : US never fabricated for
    an unregistered unit

  putStrLn "== re-registration is a hard error (C10) =="
  print (register uES (TermsVersion "dup" Map.empty) defaultStatus l6)
                                            -- Left (ReRegistration (UnitId "CME-
    ES"))

-- Example fungibility predicates (product-declared, total).
feePreserving :: FungibilityPredicate
feePreserving _ _ = Preserving   -- fee tweak within declared band: fungibility
    preserved
isinBreaking :: FungibilityPredicate
isinBreaking _ _ = Breaking      -- new ISIN: fungibility broken -> fresh unit
```

## 13 The One-Sentence Answer

> **Unit state lives in three places — immutable *ProductTerms*$[u]$, the shared *UnitStatus*$[u]$ (a projection of the log, overwritten in place as a read cache, whose every change is caused by a logged event and which replay reconstructs exactly), and per-position *PositionState*$[w, u]$ — with no separate wallet-keyed state sector, because every per-wallet economic fact is keyed by a mandate or strategy unit and so lives in *PositionState*$[w, u_{\mathrm{MA}}]$.**

This is the Pareto-optimal schema on testability, correctness, and simplicity, and the minimum basis of the problem, not a compromise.

---

*The design is the fixed point of adversarial multi-agent review; the supporting analyses are archived under* `StatesHome_work/`.